UNIT III: Software Testing Techniques

Q1. Explain White Box testing methodologies.

White Box Testing (also known as Clear Box, Glass Box, or Structural Testing) is a software testing methodology where the tester has full knowledge of the internal logic, code structure, and design of the application. The tester is "looking inside the box."

This method tests the internal pathways, data structures, and code branches rather than just the functionality (what the user sees). It is typically performed by developers or specialized QA engineers who can read the code.

Key methodologies used in White Box Testing include:

1. **Statement Coverage:** To ensure that every single executable statement in the source code has been run (or "covered") at least once.
    - **Example:** If there is an if-else block, this method would check if the statements inside the if block and the else block are executed.
    - **Formula:** (Number of statements executed / Total number of statements) * 100%
2. **Decision / Branch Coverage:** This is stronger than statement coverage. It ensures that every possible branch (or decision outcome) from a control structure (like if, case, while) is tested.
    - **Example:** For an if (A > 10) statement, the tester must write one test case where A > 10 is **True** and another test case where it is **False**.
    - **Formula:** (Number of decisions executed / Total number of decisions) * 100%
3. **Condition Coverage:** This method checks that each individual condition within a complex decision is tested as both **True** and **False**.
    - **Example:** For if (A > 10 && B < 5), this method requires tests for:
        - A > 10 is True
        - A > 10 is False
        - B < 5 is True
        - B < 5 is False
4. **Path Coverage:** This is the most thorough (and difficult) method. It aims to test every possible unique path that a program can take from its entry point to its exit point.
    - **Example:** In a program with multiple if statements, this would mean testing every combination of True/False branches.

Advantages: It is very thorough, as all code paths are tested. It can find hidden errors and inefficiencies in the code. It helps in optimizing the code.

Disadvantages: It is very complex and time-consuming. It requires skilled testers who can understand the code. It cannot find "missing features" or requirements that were never coded.

Q2. Explain Black Box testing methodologies.

Black Box Testing (also known as Behavioral or Functional Testing) is a software testing methodology where the tester has **no knowledge** of the internal code, logic, or structure of the application. The tester "cannot see inside the box."

This method focuses only on the **functionality** of the software. The tester provides inputs and observes the outputs, checking if the application behaves as specified in the requirements. This is done from an end-user's perspective.

Key methodologies (or techniques) used in Black Box Testing include:

1. **Equivalence Partitioning:** This technique involves dividing the input data into different groups (or "partitions") where all data within a group is expected to be processed in the same way.
    - The tester then picks only one test case from each partition, reducing the total number of tests.
    - *Example:* For a "Password length 8-12 chars" field, the partitions are: < 8 (Invalid), 8-12 (Valid), > 12 (Invalid).
2. **Boundary Value Analysis (BVA):** This technique is based on the idea that most errors occur at the "boundaries" (or edges) of the input partitions.
    - It involves testing the values at the minimum, maximum, just inside the boundary, and just outside the boundary.
    - *Example:* For the 8-12 char password, BVA would test: 7, 8, 9 and 11, 12, 13.
3. **Decision Table Testing:** This is used to test complex business logic where the output depends on a *combination* of different input conditions.
    - A table is created that lists all possible combinations of conditions (True/False) and the expected action for each combination.
4. **State Transition Testing:** This is used for systems that change "state" based on user actions or events.
    - It tests whether the software correctly moves from one state to another (e.g., from "Logged Out" to "Logged In").
5. **Use Case Testing:** This technique involves testing the software based on the "use cases" or "user stories" that describe how a user will interact with the system from start to finish.

Advantages: It simulates real user behavior, finding user-focused bugs. It is independent of the programming language. It can find "missing features" if the software doesn't match the requirements specification.

Disadvantages: It is not as thorough as white box testing; some code paths may never be tested. It can be difficult to design good test cases without clear requirements.

Q3. Explain Code coverage Testing.

Code Coverage (or Test Coverage) is a metric used in White Box Testing. It **measures the percentage of source code that has been executed** (or "covered") by a set of automated tests.

Code coverage is not a *type* of testing itself, but rather a **measurement of how well** the code has been tested.

Goal: The main goal of measuring code coverage is to identify parts of the software's code that have *not* been tested. A high coverage percentage gives more confidence that the code has been thoroughly checked for defects.

Types of Code Coverage Metrics:

1. **Statement Coverage:** The percentage of executable statements in the code that have been run at least once.

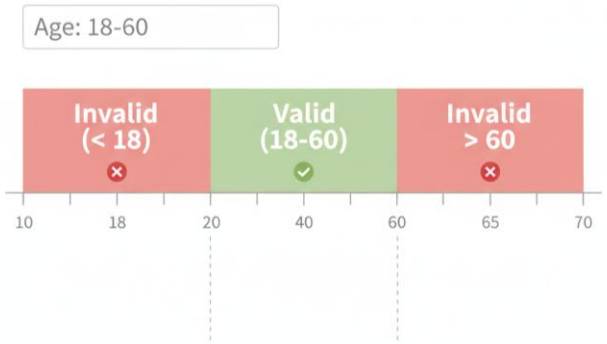
    - **Example:** If you have 100 lines of code and your tests only run 80 of those lines, you have 80% statement coverage.
2. **Decision / Branch Coverage:** The percentage of possible branches (like if-then-else) that have been executed. It checks if both the True and False paths of each decision were tested.
    - **Example:** For if (x > 5), you need one test where x > 5 is True and one where it is False to get 100% branch coverage.
3. **Condition Coverage:** The percentage of individual conditions within a complex decision that have been evaluated to both True and False.
    - **Example:** For if (A > 10 && B < 5), this measures if A>10 (T/F) and B<5 (T/F) were all tested.

How it Works: Special tools (code coverage tools) are used to run the tests. These tools monitor the code as it executes and generate a report showing which lines, branches, and conditions were (or were not) executed.
Important Note: 100% code coverage does not mean the software is 100% bug-free. It only means that all the code was executed. It's possible for a test to execute a line of code but still not find a bug (e.g., the test "expected" the wrong answer).

Q4. Explain Equivalence Partitioning.

Equivalence Partitioning (or Equivalence Class Partitioning) is a Black Box testing technique used to design test cases more efficiently.

Core Idea: The main idea is to divide all possible input data for a field into "partitions" or "classes" (groups). It is assumed that all values *within* the same partition will be processed in the exact same way by the software.



Goal: The goal is to **reduce the number of test cases** needed. Instead of testing every single possible input value (which is impossible), we only need to pick *one* representative value from each partition.

How it Works: We identify two types of partitions:
1. **Valid Partitions:** Groups of input data that the system should accept.
2. **Invalid Partitions:** Groups of input data that the system should reject.

Example: Consider a text box for "Age" that must accept values between **18 and 60**.
1. **Identify Partitions:**
    - **Partition 1 (Invalid):** Age < 18 (e.g., 10, -5)
    - **Partition 2 (Valid):** Age is between 18 and 60 (e.g., 25, 40)
    - **Partition 3 (Invalid):** Age > 60 (e.g., 65, 100)
    - **Partition 4 (Invalid):** Not a number (e.g., "ABC")

2. **Select Test Cases:** We pick one value from each partition:
    - Test Case 1: 10 (from Partition 1) -> Expected Result: Error Message
    - Test Case 2: 35 (from Partition 2) -> Expected Result: Success
    - Test Case 3: 70 (from Partition 3) -> Expected Result: Error Message
    - Test Case 4: "ABC" (from Partition 4) -> Expected Result: Error Message

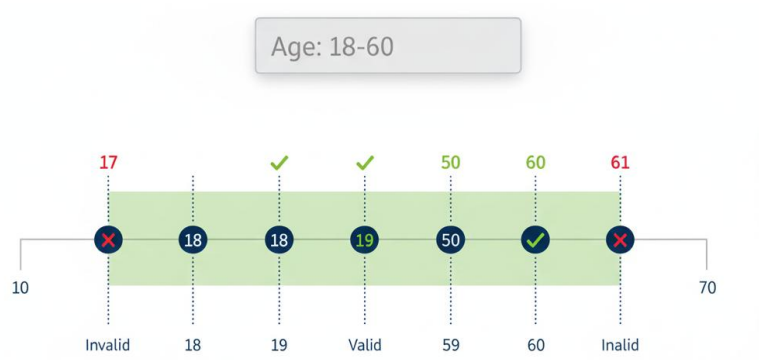
By using Equivalence Partitioning, we reduced the testing from hundreds of possibilities to just 4 simple test cases, while still covering all the "types" of input.

Q5. Explain Boundary value analysis with example.

Boundary Value Analysis (BVA) is a Black Box testing technique that is based on the idea that most software bugs occur at the "boundaries" (or "edges") of input ranges.

BVA is an extension of Equivalence Partitioning. After identifying the partitions, BVA focuses on testing the values at and around the *edges* of these partitions.

Core Idea: Instead of picking any value from the middle of a valid range (like 35), BVA checks the values at the minimum, maximum, and just inside/outside those boundaries.



How it Works: For a given input range that accepts values from Min to Max:
The test cases would be:

- **Min - 1** (Just outside the valid boundary, Invalid)
- **Min** (On the boundary, Valid)
- **Min + 1** (Just inside the boundary, Valid)
- **Max - 1** (Just inside the boundary, Valid)
- **Max** (On the boundary, Valid)
- **Max + 1** (Just outside the valid boundary, Invalid)

Example: Let's use the same "Age" field which must be between 18 and 60.

Here, Min = 18 and Max = 60. Using Boundary Value Analysis, we would create the following test cases:

- **17** (Min - 1) -> Expected Result: **Fail** (Invalid)
- **18** (Min) -> Expected Result: **Pass** (Valid)
- **19** (Min + 1) -> Expected Result: **Pass** (Valid)
- **59** (Max - 1) -> Expected Result: **Pass** (Valid)
- **60** (Max) -> Expected Result: **Pass** (Valid)
- **61** (Max + 1) -> Expected Result: **Fail** (Invalid)

This technique is very efficient because it tests the most common places for errors (like "off-by-one"

errors) using a very small number of test cases.

Q6. Explain decision table testing with example.

Decision Table Testing is a Black Box testing technique used to test systems with complex business logic or rules.

When to use: It is used when the system's output (an "Action") depends on a *combination* of several input "Conditions".

		Rule 1	Rule 2	Rule 3	Rule 4
Is the user logged in?	Yes (✓)	Yes (✓)	Yes (✓)	No (✗)	Yes (✓)
Is the shoping empy?	No (✗)	Yes (✓)	Yes (✓)		Yes (✓)
Is the shopping cart empty	No (✗)	Yes (✓)	Yes (✗)	No (✗)	Yes (✗)
Conditions		Show "Continue to Checkout" button	Show "Proceed to Checkout" button	Yes (✓) No (✗)	Show "Proceed to Checkout" button
Show "Continue Shopping" button		Yes (✓)	Yes (✗)	No (✗)	Yes (✗)
Display "Cart is Empty" message	No (✗)	Yes (✗)	No (✗)	No (✗)	Yes (✗)
Actions					

Core Components of a Decision Table: A decision table is a grid that lists all possible combinations of conditions and the actions that should be taken for each combination.

1. **Conditions (Stub):** The list of all input conditions that affect the logic (e.g., "Is user logged in?").
2. **Actions (Stub):** The list of all possible outputs or actions of the system (e.g., "Grant access").
3. **Rules (Columns):** Each column in the table represents a "Rule," which is a unique combination of True/False for the conditions and the resulting action.

Example: Let's test a "Login" feature.

Business Rules:

- If the username and password are correct, log the user in.
- If the username is wrong, show "Invalid Username" error.
- If the username is correct but the password is wrong, show "Invalid Password" error.

1. Identify Conditions and Actions:

- **Conditions:**
  - C1: Is Username valid? (T/F)
  - C2: Is Password valid? (T/F)
- **Actions:**
  - A1: Show "Login Successful"
  - A2: Show "Invalid Username"
  - A3: Show "Invalid Password"

2. Create the Decision Table: We list all possible T/F combinations for the conditions.

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
C1: Username Valid?	T	T	F	F

C2: Password Valid?	T	F	T	F
Actions				
A1: Show "Login Successful"	Yes	-	-	-
A2: Show "Invalid Username"	-	-	Yes	Yes
A3: Show "Invalid Password"	-	Yes	-	-

3. Create Test Cases: This table gives us 4 test cases:

- **TC1 (Rule 1):** Enter valid username AND valid password. -> Expected: "Login Successful".
- **TC2 (Rule 2):** Enter valid username AND invalid password. -> Expected: "Invalid Password".
- **TC3 (Rule 3):** Enter invalid username AND valid password. -> Expected: "Invalid Username".
- **TC4 (Rule 4):** Enter invalid username AND invalid password. -> Expected: "Invalid Username".

Q7. Explain mutation testing.

Mutation Testing is a powerful White Box testing technique that is used to evaluate the **quality and effectiveness of your test cases**.

The main idea is not to test the software's code, but to test the **test suite itself**. It checks if your tests are good enough to find small bugs.

How it Works (The Process):

1. **Start:** You begin with a piece of code (the "original") and a set of test cases that all pass (a "green" test suite).
2. **Mutate:** The mutation testing tool automatically creates many copies of the original code. In each copy, it makes one tiny change (a "mutation"). This new, faulty version of the code is called a **"Mutant"**.
    - *Examples of Mutations:* Changing + to -, Changing a > b to a >= b, Changing True to False, Deleting a line of code
3. **Test the Mutant:** The tool then runs your *original* test suite against this new "Mutant" code.
4. **Analyze the Result:**
    - **Case 1: The Mutant is "Killed"**
        - One of your test cases *fails*.
        - This is **GOOD!** It means your test suite was strong enough to "catch" the small bug (the mutation) that was introduced.
    - **Case 2: The Mutant "Survives"**
        - All of your test cases *pass*, even on the faulty mutant code.
        - This is **BAD!** It means your test suite has a "gap" and is not good enough to find this specific type of bug. The tests need to be improved.

Goal: The goal of mutation testing is to achieve a high "Mutation Score" (the percentage of mutants killed). A high score means your test suite is robust and high-quality.

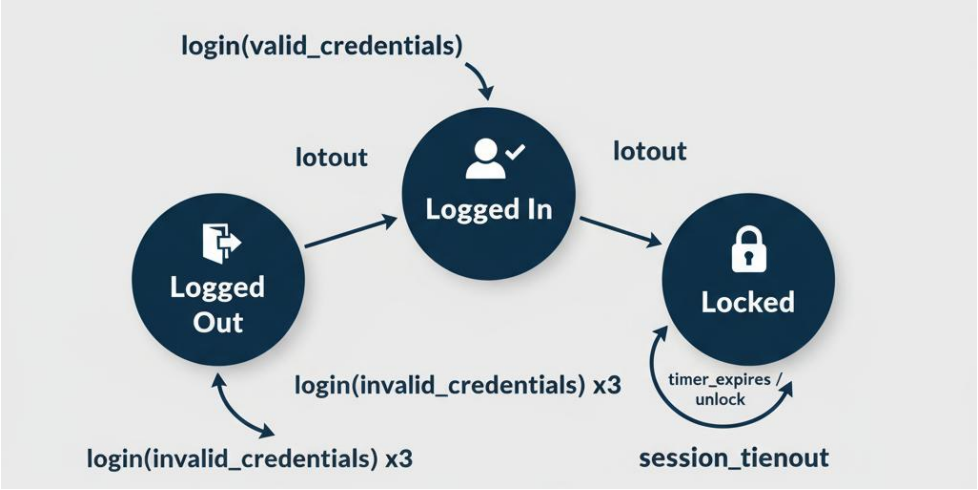
Disadvantage: Mutation testing is very computationally expensive and slow, as it requires running the entire test suite hundreds or thousands of times (once for each mutant).

Q8. What is State Transition Testing?

State Transition Testing is a Black Box testing technique used to test the behavior of a system that can exist in different "states."

Core Concepts:

1. **State:** A "state" is a specific condition or status of the software at a given time.
   - *Example:* A user login system can be in the "Logged In" state or the "Logged Out" state. An ATM can be in an "Idle" state, "Card Inserted" state, or "PIN Entered" state.
2. **Transition:** A "transition" is the *event* or *action* that causes the software to move from one state to another.
   - *Example:* The event "Enter correct password" causes a transition from the "Logged Out" state to the "Logged In" state.



How it Works: The tester performs the following steps:

1. **Identify States:** List all the possible states the system can be in.
2. **Identify Transitions:** List all the events (inputs or actions) that can cause the system to change state.
3. **Create a Diagram:** The tester often draws a **State Transition Diagram** (a flowchart) to visualize all the states and the valid transitions between them.
4. **Write Test Cases:** Test cases are written to cover:
   - **Valid Transitions:** Testing every valid path from one state to another (e.g., test that a correct login moves you to the "Logged In" state).
   - **Invalid Transitions:** Testing events that should *not* cause a state change (e.g., trying to access "My Account" when in the "Logged Out" state).
   - **Error States:** Testing transitions that lead to error states (e.g., entering the wrong password 3 times transitions to a "Locked" state).

Example (Login System):

- **States:** Logged Out, Logged In, Locked
- **Events:** valid_pass, invalid_pass, logout, invalid_pass_3_times
- **Test Cases:**
  - From Logged Out, use valid_pass -> Check state is Logged In.
  - From Logged In, use logout -> Check state is Logged Out.
  - From Logged Out, use invalid_pass -> Check state is still Logged Out.
  - From Logged Out, use invalid_pass_3_times -> Check state is Locked.